



Facultad de Ciencias Exactas y Naturales y Agrimensura

Paradigmas y Lenguajes de Programación en Lenguaje LISP

**TRABAJO PRACTICO INTEGRADOR 2026: Sistema de
Semáforos Inteligentes y Análisis Comparativo de
Paradigmas.**

Integrantes – Grupo 32:

- Gauna, Gaston Leonel.
- Muñoz, Ernestina del Carmen.
- Oviedo Bedoya, Luis.
- Rojas, Vera.

Tema Investigado: Lenguaje de Programación Clojure.

Fundamentos del Diseño Funcional Adoptado

El sistema fue desarrollado siguiendo los principios de la programación funcional, priorizando la modularidad, la reutilización de código y la separación de responsabilidades. Cada función fue diseñada para cumplir una tarea específica y luego integrarse con las demás para resolver los distintos requerimientos.

Requerimientos 1 y 2: Estados de Transición y Temporizador Automático

La función **transicion** controla los cambios permitidos entre los estados del semáforo. Recibe el color actual y el color destino, y mediante una estructura **cond** verifica si la transición es válida. Con la incorporación de la Extensión 1 se agregaron estados intermitentes de seguridad, permitiendo modelar los 3 segundos de transición entre cambios de color.

La función **timer** determina automáticamente el estado del semáforo según el tiempo transcurrido. Utiliza `(mod timestamp 225)` para obtener la posición dentro de un ciclo de 225 segundos y, mediante condiciones, asigna el color correspondiente a cada intervalo de tiempo.

Mientras **timer** define el estado según el tiempo, **transicion** valida los cambios entre estados, trabajando ambas funciones de manera complementaria.

Requerimiento 3 y Extensión 2: Sistema de Auditoria

Este requerimiento se resolvió reutilizando las funciones desarrolladas anteriormente mediante la expresión:

```
(transicion          color-actual          (timer          tiempo))
```

La Extensión 2 incorporó la generación de registros en un archivo de texto. Para ello se procesan listas con el formato `(tiempo color-actual)` utilizando funciones de acceso como `car` y `cadr`.

Además, se implementaron las funciones **cambiarNombre** y **formatoColor** para convertir los resultados de las transiciones a un formato más legible. La generación de fechas y horas se realizó mediante la librería `local-time`, mientras que el procesamiento de múltiples registros se implementó con `mapcar` y `lambda`.

Requerimiento 4: Análisis de Ciclos Semafóricos

Este requerimiento calcula la cantidad de ciclos completos transcurridos a partir de un tiempo dado y una duración de ciclo recibida como parámetro. Se utilizó la función `floor` para obtener únicamente los ciclos completos.

Además, se incorporó una evaluación que clasifica la duración del ciclo como demasiado corta, demasiado larga o adecuada, proporcionando una recomendación al usuario.

Requerimiento 5: Planificación Temporal

La función **ciclos-por-tiempo** determina cuántos ciclos completos realiza el semáforo en una cantidad determinada de minutos.

Primero convierte los minutos a segundos y luego divide ese valor por la duración total de un ciclo (225 segundos). Finalmente utiliza `floor` para descartar fracciones de ciclo y devolver únicamente ciclos completos.

Requerimiento 6: Informe de Distribución Temporal

El objetivo fue calcular el porcentaje de tiempo que cada color permanece activo durante una hora de funcionamiento.

Para ello se determinó la cantidad de ciclos completos en una hora, se acumularon los tiempos correspondientes a cada estado y luego se calculó el porcentaje de participación de cada color utilizando la fórmula:

$$\text{Porcentaje} = (\text{Tiempo del color} / \text{Tiempo total}) \times 100$$

De esta manera se obtuvo un informe estadístico que permite analizar la distribución temporal de los estados del semáforo.

Requerimiento 7: Aseguramiento de la Calidad

Casos de Prueba del Requerimiento 1: Función transicion.

Se realizaron pruebas para verificar que la función efectúe correctamente los cambios de estado permitidos entre las luces del semáforo.

Entradas Evaluadas:

(transicion	'en-rojo	'rojo-intermitente)
(transicion	'rojo-intermitente	'verde)
(transicion	'en-verde	'verde-intermitente)
(transicion	'verde-intermitente	'amarillo)

```
(transicion      'en-amarillo      'amarillo-intermitente)
(transicion      'amarillo-intermitente      'rojo)
(transicion      'en-rojo      'amarillo)
(transicion 'azul 'verde)
```

Resultados obtenidos:

```
(EN-ROJO "cambiar-a-rojo-intermitente")
(ROJO-INTERMITENTE "cambiar-a-verde")
(EN-VERDE "cambiar-a-verde-intermitente")
(VERDE-INTERMITENTE "cambiar-a-amarillo")
(EN-AMARILLO "cambiar-a-amarillo-intermitente")
(AMARILLO-INTERMITENTE "cambiar-a-rojo")
(EN-ROJO ACCION-POR-DEFECTO)
(AZUL ACCION-POR-DEFECTO)
```

Análisis:

Los resultados muestran que la función reconoce correctamente todas las transiciones válidas del ciclo semafórico. Cuando se solicita una transición no permitida o se utiliza un estado inexistente, la función devuelve la acción por defecto, evitando comportamientos inesperados.

Casos de Prueba del Requerimiento 2: Función timer

La función determina el estado correspondiente del semáforo según el segundo transcurrido dentro del ciclo completo.

Entradas evaluadas:

```
(timer 0)
(timer 89)
(timer 91)
(timer 93)
(timer 150)
(timer 214)
```

```
(timer 220)
(timer 223)
(timer 225)
(timer 450)
(timer 'hola)
```

Resultados obtenidos:

ROJO

ROJO

ROJO-INTERMITENTE

VERDE

VERDE

VERDE-INTERMITENTE

AMARILLO

AMARILLO-INTERMITENTE

ROJO

ROJO

Error de tipo: HOLA no es un número válido para la operación MOD.

Análisis:

Las pruebas verifican que la función identifica correctamente cada estado del ciclo, incluyendo los estados intermitentes incorporados en la segunda iteración. También se comprobó el comportamiento ante datos inválidos, generando un error cuando se recibe un valor que no es numérico.

Casos de Prueba del Requerimiento 3 y la Extensión 2:

Se realizaron simulaciones utilizando distintos tiempos y estados iniciales para verificar la coherencia entre las funciones implementadas.

Entradas Evaluadas:

(10 en-rojo)

(40 en-rojo)
(100 en-verde)
(150 en-verde)
(45 en-amarillo-intermitente)
(91 en-rojo)
(150 en-rojo-intermitente)
(214 en-verde)
(218 en-verde-intermitente)
(224 en-amarillo)

Resultado Obtenido:

Informe de Ejecución del Sistema Semafórico

=====

2026-06-14T10:18:11.000000Z - Transición: ROJO -> ROJO
2026-06-14T10:18:42.000000Z - Transición: ROJO -> ROJO
2026-06-14T10:19:42.000000Z - Transición: VERDE -> VERDE
2026-06-14T10:20:32.000000Z - Transición: VERDE -> VERDE
2026-06-14T10:18:47.000000Z - Transición: AMARILLO-INTERMITENTE -
> ROJO
2026-06-14T10:19:33.000000Z - Transición: ROJO -> ROJO-
INTERMITENTE
2026-06-14T10:20:32.000000Z - Transición: ROJO-INTERMITENTE ->
VERDE
2026-06-14T10:21:36.000000Z - Transición: VERDE -> VERDE-
INTERMITENTE
2026-06-14T10:21:40.000000Z - Transición: VERDE-INTERMITENTE ->
AMARILLO
2026-06-14T10:21:46.000000Z - Transición: AMARILLO -> AMARILLO-
INTERMITENTE

--- Fin del Informe ---

Análisis:

La simulación demuestra que el sistema realiza correctamente las transiciones entre estados, respetando la duración establecida para cada color y los períodos de intermitencia de seguridad.

Casos de Prueba del Requerimiento 4: Función duracion-ciclo:

Esta función permite analizar si la cantidad de ciclos programados para una hora es adecuada.

Entradas Evaluadas:

(duracion-ciclo 3600 20)

(duracion-ciclo 3600 90)

(duracion-ciclo 3600 180)

Resultados Obtenidos:

En 3600 segundos se realizan 20 ciclos completos.

El ciclo es demasiado largo. Disminuir la duración del ciclo.

En 3600 segundos se realizan 180 ciclos completos.

El ciclo es muy corto. Aumentar la duración del ciclo.

En 3600 segundos se realizan 40 ciclos completos.

El ciclo está en la duración óptima.

Análisis:

La función permite evaluar rápidamente si la configuración seleccionada produce una frecuencia de cambios adecuada para el funcionamiento del semáforo.

Casos de Prueba del Requerimiento 5: Función ciclos-por-tiempo:

Esta función calcula la cantidad de ciclos completos que pueden realizarse en un período determinado.

Entradas Evaluadas:

(ciclos-por-tiempo 3.75)

(ciclos-por-tiempo 10)

(ciclos-por-tiempo 60)

Resultados Obtenidos:

1 ciclo entero.

2 ciclos enteros.

16 ciclos enteros.

Análisis:

La función utiliza floor para considerar únicamente ciclos completos, descartando las fracciones de ciclo que no llegan a finalizarse dentro del período analizado.

Casos de Prueba del Requerimiento 6: Función informe-por-hora

Se calculó el porcentaje de tiempo que ocupa cada estado dentro de un ciclo completo de 225 segundos.

Resultado Obtenido:

ROJO 40,00 %

ROJO-INTERMITENTE 1,33 %

VERDE 53,33 %

VERDE-INTERMITENTE 1,33 %

AMARILLO 2,67 %

AMARILLO-INTERMITENTE 1,33 %

Análisis:

La distribución muestra que la mayor parte del tiempo corresponde al estado verde, favoreciendo la circulación vehicular. Los estados intermitentes representan una pequeña porción del ciclo total y cumplen la función de transición segura entre colores.

Justificación de la Selección de Librerías: Fase 2

Para el desarrollo de esta fase, se optó por integrar al núcleo de la aplicación la librería externa **local-time** a través del gestor de paquetes Quicklisp, descartando la alternativa **cl-json**. Esta decisión se fundamenta en su alineación directa con los objetivos del proyecto y en principios de eficiencia en ingeniería de software. En primer lugar, la arquitectura del sistema exige explícitamente en el Sistema de Auditoría y en la Extensión de Persistencia el registro de eventos en un formato cronológico legible para humanos (\$AAAA-MM-DD\$ \$HH:MM:SS\$). La librería elegida resuelve esta necesidad de manera nativa mediante funciones optimizadas de abstracción y formateo temporal, transformando el tiempo Unix en una sola línea de código.

Por el contrario, prescindir de esta herramienta obligaría a desarrollar algoritmos matemáticos complejos desde cero para traducir el tiempo universal de Common Lisp al calendario gregoriano, teniendo que gestionar manualmente variables como los años bisiestos, la duración dispar de los meses y las zonas horarias. Delegar esta lógica en un componente maduro del ecosistema minimiza el riesgo de introducir errores de cálculo y reduce la complejidad del código. Finalmente, se desestimó el uso de **cl-json** debido a que las especificaciones del problema ya definen de forma estática los tiempos de los ciclos semafóricos como reglas de negocio preestablecidas. Introducir un archivo de configuración externa JSON añadiría una capa de abstracción innecesaria para el alcance actual del sistema, mientras que **local-time** optimiza y eleva la calidad técnica de un subsistema crítico y ya solicitado.

Bitácora Pormenorizada de Bugs de Depuración

Bug #1: Error de unidades.

Descripción: Al probar (ciclos-por-tiempo 10), la función devolvía 0 en lugar de 2.

Causa: Olvido conceptual de la conversión de unidades, se dividían los minutos directamente por la constante de segundos (10/225), pero al dar decimales (0.04), floor lo redondeaba a 0.

Solución: Se agregó la multiplicación (*minutos 60) en el let para transformar el tiempo a segundos antes de dividir.

Bug #2: Error de sintaxis.

Descripción: El código no cargaba en el entorno y tiraba error de variable no ligada (unbound variable).

Causa: Típico error de tipeo con los paréntesis. Se cerró el bloque del let justo después de declarar la variable, dejando la función floor colgada afuera del cuerpo del let.

Solución: Se corrigió el orden de los paréntesis para que la operación aritmética quede dentro de la estructura del let.

Bug #3: Descoordinación de constantes (216 seg vs 225 seg).

Descripción: Al testear con una hora (ciclos-por-tiempo 60), mi función tiraba 16 ciclos, pero los resultados no coinciden con la función duración-ciclo de mis compañeros.

Causa: Falta de comunicación grupal. Yo seguía usando la constante inicial de 216 segundos, sin registrar que el grupo había extendido el ciclo a 225 segundos en el timer para incluir los estados intermitentes.

Solución: Se actualizó el divisor fijo de 216 a 225 para unificar los criterios con el código final del equipo.

Bug #4: Descoordinación en la duración total del ciclo.

Descripción: Al calcular la cantidad de ciclos en una hora completa, el resultado no era exacto. Con la versión inicial del sistema, que solo contemplaba los estados rojo, verde y amarillo, se obtenían 16 ciclos completos más un remanente de segundos que iniciaba un nuevo ciclo. Como solución temporal, se consideraron únicamente los 16 ciclos completos, ignorando los segundos sobrantes.

Causa: El diseño original no incluía los estados intermitentes entre los cambios de color. Debido a ello, la duración total del ciclo no era un divisor exacto de una hora, generando un tiempo residual al finalizar los 16 ciclos.

Solución: Se incorporaron los estados intermitentes de seguridad, con una duración de 3 segundos entre cada transición de color. Este ajuste modificó la duración total del ciclo, logrando que una hora se dividiera exactamente en 16 ciclos completos, sin sobrantes de tiempo y manteniendo la coherencia con la versión final del sistema.

Clojure: características, aplicaciones y utilización en la industria.

Introducción

Clojure es un lenguaje de programación moderno perteneciente a la familia de los lenguajes Lisp. Fue creado por Rich Hickey y presentado oficialmente en 2007 con el objetivo de combinar la simplicidad y expresividad características de Lisp con las necesidades de los sistemas informáticos modernos. Una de sus principales particularidades es que se ejecuta sobre la Máquina Virtual de Java (JVM), lo que le permite aprovechar todo el ecosistema de bibliotecas y herramientas desarrolladas para

Java. Además, existen implementaciones que permiten utilizar Clojure en otros entornos, como JavaScript mediante ClojureScript.

Desde su creación, Clojure ha ganado reconocimiento por promover un paradigma de programación funcional, donde las funciones puras, la inmutabilidad de los datos y la ausencia de efectos secundarios son conceptos fundamentales. Estas características facilitan el desarrollo de aplicaciones robustas, mantenibles y capaces de manejar altos niveles de concurrencia, un requisito cada vez más importante en los sistemas actuales.

Utilización de Clojure en la industria

Aunque no posee la popularidad masiva de otros lenguajes como Java, Python o JavaScript, Clojure ocupa un lugar destacado en organizaciones que requieren sistemas altamente confiables y escalables. Su diseño orientado a la programación funcional permite reducir la complejidad del software y minimizar errores relacionados con el manejo de estados compartidos.

Las aplicaciones desarrolladas en Clojure se encuentran principalmente en sectores tecnológicos avanzados, servicios financieros, plataformas de comercio electrónico, procesamiento de grandes volúmenes de datos, sistemas de monitoreo y aplicaciones distribuidas. Gracias a su integración con Java, las empresas pueden incorporar Clojure dentro de infraestructuras existentes sin necesidad de reemplazar completamente sus sistemas.

Diversas compañías reconocidas han adoptado Clojure como parte de sus tecnologías de desarrollo. Entre ellas se encuentra Walmart, que utiliza el lenguaje para gestionar grandes cantidades de información provenientes de miles de tiendas físicas y plataformas digitales. También destaca Nubank, una de las fintech más importantes de América Latina, que emplea Clojure en gran parte de su infraestructura tecnológica debido a su capacidad para desarrollar sistemas financieros seguros y escalables.

Otra empresa destacada es Atlassian, conocida por herramientas como Jira y Confluence, que ha utilizado Clojure en proyectos relacionados con la colaboración en tiempo real. Del mismo modo, GO-JEK, una de las plataformas digitales más grandes del sudeste asiático, recurrió a Clojure para optimizar sistemas que deben atender millones de solicitudes de usuarios diariamente. Empresas como Funding Circle, The Climate Corporation y Consumer Reports también han reportado casos exitosos utilizando este lenguaje en aplicaciones críticas para sus operaciones.

Estos ejemplos demuestran que Clojure no solo posee valor académico, sino que también es una herramienta utilizada en entornos productivos de gran escala donde la confiabilidad y el rendimiento son factores esenciales.

Keywords y representación de estados en Clojure

Uno de los aspectos que diferencia a Clojure de otros dialectos de Lisp es el uso de Keywords para representar identificadores y estados. En los Lisp tradicionales es habitual utilizar símbolos citados, por ejemplo 'rojo, 'amarillo o 'verde. En cambio, Clojure introduce los Keywords, representados mediante el prefijo dos puntos: :rojo, :amarillo y :verde.

La principal ventaja técnica de los Keywords es que son autoevaluables. Esto significa que su valor es el propio keyword y no necesitan ser buscados dentro de un entorno de variables para determinar su significado. Como consecuencia, el lenguaje puede procesarlos de forma más eficiente y con menor posibilidad de errores.

Además, los Keywords son ampliamente utilizados como claves dentro de mapas, una de las estructuras de datos más importantes de Clojure. Por ejemplo, un estado de semáforo puede representarse mediante un mapa como:

```
{:estado :rojo}
```

Esta representación resulta clara, legible y consistente con el estilo de programación funcional promovido por el lenguaje.

Desde el punto de vista del rendimiento, los Keywords presentan una ventaja adicional. Clojure mantiene una única instancia de cada keyword en memoria, proceso conocido como internamiento. Gracias a ello, las comparaciones entre keywords son extremadamente rápidas y consumen menos recursos que otras alternativas. En un simulador semafórico donde constantemente se comparan estados como rojo, amarillo y verde, el uso de Keywords permite obtener un código más eficiente y seguro.

Manejo del tiempo mediante estructuras inmutables

Otra característica fundamental de Clojure es que las estructuras de datos son inmutables por defecto. Una vez creado un dato, este no puede modificarse directamente. Aunque inicialmente esto puede parecer una limitación, en realidad constituye una de las fortalezas más importantes del lenguaje.

En muchos lenguajes de programación es común representar el paso del tiempo modificando variables. Por ejemplo, una variable que almacena el estado de un semáforo podría cambiar sucesivamente de rojo a verde y luego a amarillo. Sin embargo, en Clojure este enfoque no es el habitual. En lugar de alterar el valor existente, se crea una nueva versión del estado cada vez que ocurre una transición.

De esta manera, si un semáforo se encuentra en estado rojo, una función puede recibir dicho estado como entrada y devolver un nuevo estado verde como resultado. El valor original permanece intacto y el sistema continúa trabajando con la nueva versión generada.

Este comportamiento es posible gracias al uso de estructuras de datos persistentes. Dichas estructuras permiten compartir internamente gran parte de la información entre versiones sucesivas, evitando copias completas y reduciendo significativamente el consumo de memoria. Como consecuencia, se obtiene la seguridad de la inmutabilidad sin sacrificar eficiencia.

En un simulador de semáforos, el paso del tiempo puede modelarse como una secuencia de transformaciones de estado. Una función recibe un estado determinado y devuelve el siguiente según las reglas establecidas. Por ejemplo, un estado rojo genera un estado verde, un estado verde genera un estado amarillo y un estado amarillo genera nuevamente un estado rojo. El sistema avanza a través de estos estados sucesivos sin necesidad de modificar variables existentes.

Este enfoque aporta importantes ventajas. Al no existir cambios directos sobre los datos, se reducen los errores asociados a modificaciones inesperadas, se facilita el razonamiento sobre el comportamiento del programa y se mejora el manejo de procesos concurrentes. Estas características son especialmente valiosas en aplicaciones donde múltiples componentes interactúan simultáneamente.

Conclusión

Clojure representa una evolución moderna de los lenguajes Lisp, combinando la expresividad característica de esta familia con herramientas diseñadas para afrontar los desafíos del desarrollo de software contemporáneo. Su énfasis en la programación funcional, la inmutabilidad y la concurrencia lo convierten en una opción sólida para la construcción de sistemas complejos y escalables.

La adopción de Clojure por parte de empresas como Walmart, Nubank, Atlassian, GOMER y otras organizaciones demuestra que se trata de una tecnología madura y capaz de responder a exigencias reales de la industria. En el contexto de un simulador de semáforos, los Keywords proporcionan una forma eficiente de representar estados, mientras que las estructuras de datos persistentes permiten modelar el paso del tiempo mediante la creación de nuevos estados en lugar de modificar los existentes. Estas características reflejan la filosofía central de Clojure y explican gran parte de su éxito en aplicaciones modernas.

Bibliografía

<https://clojure.org/>

https://clojure.org/community/success_stories

<https://es.wikipedia.org/wiki/Clojure>